

PERTEMUAN 7

CLASS (LANJUTAN)

7.1 Inheritance

Inheritance merupakan salah satu point penting dalam pembuatan pemrograman berorientasi objek. Sebuah class atau objek dapat saling berhubungan dengan class yang lain, salah satu bentuk hubungannya adalah *inheritance* (pewarisan). Hubungan ini seperti hubungan keluarga antara orang tua dan anak. Contohnya orangtua yang baik, dapat menurunkan anak yang baik pula, begitupun dalam kodingan. Sebuah class di Java, dapat memiliki satu atau lebih keturunan atau class anak (subclass). Class anak akan memiliki warisan atribut dan method dari class ibu.

Contoh didalam package Hewan membuat class-class nama hewan dengan perilaku yang berbeda, lalu buat kode untuk masing-masing class:

Zebra

```
class Zebra {  
    String name;  
    int age;  
    int speed;  
  
    void walk(){  
        // ...  
    }  
    void run(){  
        //...  
    }  
}
```

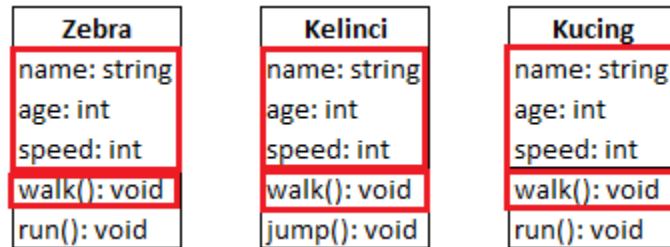
Kelinci

```
class Kelinci {  
    String name;  
    int age;  
    int speed;  
  
    void walk(){  
        // ...  
    }  
    void jump(){  
        //...  
    }  
}
```

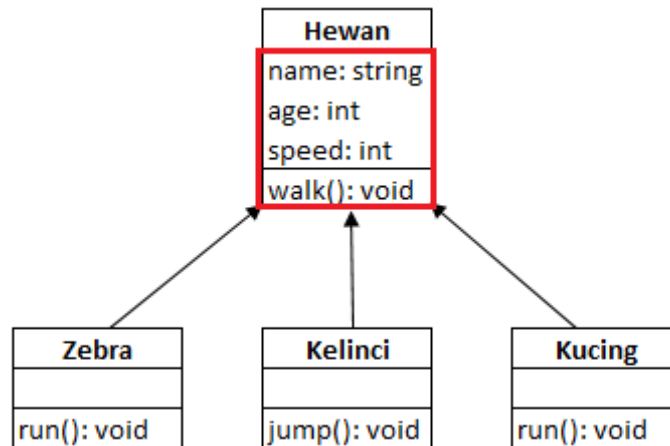
Kucing

```
class Kucing {  
    String name;  
    Int age;  
    int speed;  
  
    void walk(){  
        // ...  
    }  
    void run(){  
        //...  
    }  
}
```

Daripada menulis berulang-ulang properti dan method yang sama, dapat meningkatnya dengan menggunakan *inheritance*. Cari member class yang sama:



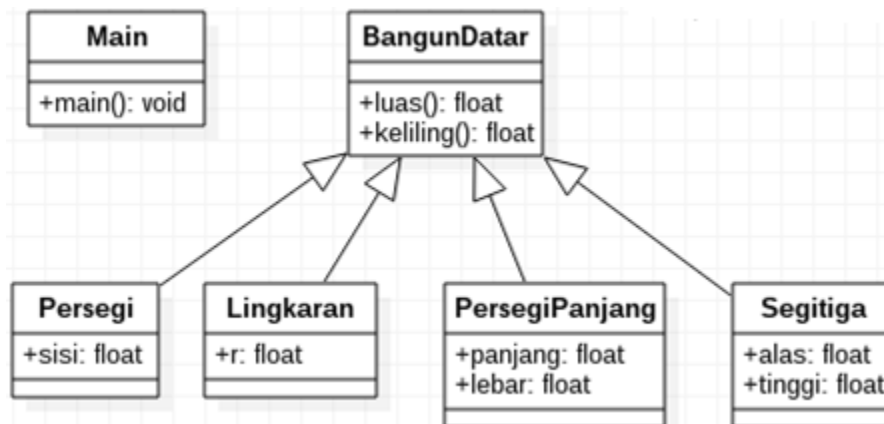
Setelah menggunakan *inheritance*, maka akan menjadi seperti ini:



Hewan adalah class induk yang memiliki class anak Zebra, Kelinci, dan Kucing. Apapun properti atau atribut yang ada di class induk, akan dimiliki juga oleh class anak.

11.2 Membuat Program Inheritance

Setelah memahami konsep *inheritance*, buat contoh program sederhana. Program yang dibuat berfungsi untuk menghitung luas dan keliling bangun datar. Bentuk class diagramnya:



1. Buka Netbeans, buat *Java Package* baru bernama **inheritance** di dalam **Source Packages**.
2. Isi nama *package* dengan **inheritance**.

3. Buat class-class baru berdasarkan diagram di atas:

>> bangundatar.java <<

```
package inheritance;

public class bangundatar {
    float luas() {
        System.out.println("Menghitung luas bangun datar");
        return 0;
    }

    float keliling() {
        System.out.println("Menghitung keliling bangun datar");
        return 0;
    }
}
```

>> persegi.java <<

```
package inheritance;

public class persegi extends bangundatar {
    float sisi;
}
```

>> lingkaran.java <<

```
package inheritance;

public class lingkaran extends bangundatar {
    // jari-jari lingkaran
    float r;
}
```

>> persegipanjang.java <<

```
package inheritance;

public class persegipanjang extends bangundatar {
    float panjang;
    float lebar;
}
```

>> segitiga.java <<

```
package inheritance;

public class segitiga extends bangundatar {
    float alas;
    float tinggi;
}
```

>> mainbangundatar.java <<

```
package inheritance;

public class mainbangundatar {
    public static void main(String[] args) {
        // membuat objek bangun datar
        bangundatar mBangunDatar = new bangundatar();
        // membuat objek persegi dan mengisi nilai properti
        persegi mPersegi = new persegi();
        mPersegi.sisi = 2;
        // membuat objek Lingkaran dan mengisi nilai properti
        lingkaran mLingkaran = new lingkaran();
        mLingkaran.r = 22;
        // membuat objek Persegi Panjang dan mengisi nilai properti
        persegipanjang mPersegiPanjang = new persegipanjang();
        mPersegiPanjang.panjang = 8;
        mPersegiPanjang.lebar = 4;
        // membuat objek Segitiga dan mengisi nilai properti
        segitiga mSegitiga = new segitiga();
        mSegitiga.alas = 12;
        mSegitiga.tinggi = 8;
    }
}
```

```

// memanggil method luas dan keliling
mBangunDatar.luas();
mBangunDatar.keliling();

mPersegi.luas();
mPersegi.keliling();

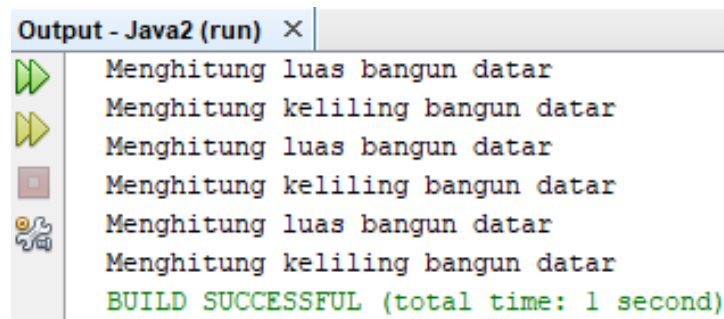
mLingkaran.luas();
mLingkaran.keliling();

mPersegiPanjang.luas();
mPersegiPanjang.keliling();

mSegitiga.luas();
mSegitiga.keliling();
}
}

```

Jalankan **mainbangundatar.java**, akan didapatkan hasil seperti dibawah ini:



```

Output - Java2 (run) X
Menghitung luas bangun datar
Menghitung keliling bangun datar
Menghitung luas bangun datar
Menghitung keliling bangun datar
Menghitung luas bangun datar
Menghitung keliling bangun datar
BUILD SUCCESSFUL (total time: 1 second)

```

Mengapa hasilnya seperti diatas? Bukan hasil berupa angka? Inilah peran method Overriding, dari sini akan dilanjutkan dengan menggunakan method Overriding.

11.3 Method Overriding

Method Overriding dilakukan saat ingin membuat ulang sebuah method pada subclass atau class anak. Method Overriding dapat dibuat dengan menambahkan anotasi `@Override` di atas nama method atau sebelum pembuatan method.

1. Buka kembali file **persegi.java**, lalu tambahkan **@Override** dibawah tipe data float, untuk jelasnya dapat dilihat pada codingan dibawah ini:

```
package inheritance;
```

```
class persegi extends bangundatar {  
    float sisi;  
    @Override  
    float luas() {  
        float luas = sisi * sisi;  
        System.out.println("Luas Persegi: " + sisi + " x " + sisi + " = " + luas);  
        return luas;  
    }  
    @Override  
    float keliling() {  
        float keliling = 4 * sisi;  
        System.out.println("Keliling Persegi: 4 x " + sisi + " = " + keliling);  
        return keliling;  
    }  
}
```

2. Buka kembali file **lingkaran.java**, tambahkan juga **@Override** seperti kodingan dibawah ini:

```
package inheritance;
```

```
public class lingkaran extends bangundatar {  
    // jari-jari lingkaran (pi = 3.14)  
    float r;  
    @Override  
    float luas() {  
        float luas = (float) (Math.PI * r * r);  
        System.out.println("Luas lingkaran: 3.14 x "+r+" x "+r+" = "+luas);  
        return luas;  
    }  
    @Override  
    float keliling() {  
        float keliling = (float) (2 * Math.PI * r);  
        System.out.println("Keliling Lingkaran: 2 x 3.14 x "+r+" = "+keliling);  
        return keliling;  
    }  
}
```

Dalam rumus luas dan keliling lingkaran, dapat menggunakan konstanta **Math.PI** sebagai nilai **PI**. Konstanta ini sudah ada di Java. Fungsi **@Override** yaitu menulis ulang method **luas()** dan **keliling()** pada class anak, jadi inheritance dan **@Override** ini saling berhubungan satu dengan yang lainnya.

Latihan

1. Pada pertemuan 7 dengan contoh program Bangun Datar menghitung luas dan keliling, terdiri dari lingkaran, persegi, persegi panjang, dan segitiga. Pada contoh program sudah ditambahkan @Override file lingkaran.java dan persegi.java, lanjutkan kembali program mencari luas dan keliling:

- persegipanjang.java → NIM ganjil

- segitiga.java → NIM genap

Setelah itu, jalankan kembali program class **mainbangundatar.java**!

2. Screenshot kodingan program dan hasil program. Di dalam kodingan program, beri nama masing-masing (contoh: //galuhsaputri).